

Proyecto Final: Analizador de Comentarios Turísticos con Visualizaciones Interactivas

Objetivo General

Desarrollar un script en Python, ejecutable desde la línea de comandos, que permita analizar comentarios turísticos contenidos en un archivo CSV. El script deberá realizar tareas de procesamiento de lenguaje natural (PLN), limpieza de datos, análisis de sentimientos, modelado de tópicos y generación de visualizaciones interactivas. Todo el proceso deberá ser autónomo (sin uso de APIs externas) y estar documentado para su fácil instalación y ejecución en cualquier entorno.

Requerimientos Funcionales (Entrada)

El script deberá aceptar los siguientes parámetros obligatorios por línea de comandos:

1. **Ruta del archivo CSV** que contiene los datos.
2. **Nombre de la columna** dentro del CSV donde se encuentran los comentarios textuales.
3. **Idioma objetivo** del análisis (ej. **es**, **en**, **fr**). Este parámetro afectará los modelos de tokenización, stopwords y stemming/lematización.
4. **Título del reporte** que se usará en la generación de las visualizaciones y/o archivos de salida.
5. **Paleta de colores** a utilizar en las gráficas. Debe incluir al menos una opción que respete criterios de accesibilidad visual (accesible para personas con daltonismo). Se recomiendan paletas como **viridis**, **cividis**, **plasma** o **inferno**.

Requerimientos de Procesamiento

El script deberá ejecutar el siguiente flujo de análisis sobre los comentarios:

1. **Limpieza y preprocesamiento de texto:**
 - Eliminación de caracteres especiales, URLs, números (si no son relevantes), mayúsculas/minúsculas, etc.

- Tokenización, eliminación de stopwords y aplicación de stemming o lematización según el idioma.
2. **Detección y análisis de valores atípicos (outliers):**
- Identificar comentarios atípicos mediante alguna de las técnicas vistas en clase.
 - **Sobre los comentarios etiquetados como outliers** se generará un análisis de:
 - Unigramas
 - Bigramas
 - Trigramas
 - Los comentarios **no atípicos (normales)** continuarán al siguiente paso del análisis.
3. **Análisis de sentimientos:**
- Clasificar cada comentario como **positivo** o **negativo** (puede ser una clasificación binaria o basada en umbral de polaridad).
 - Separar los comentarios en dos grupos: positivos y negativos.
4. **Modelado de tópicos por sentimiento:**
- Para los comentarios positivos, aplicar modelado de tópicos.
 - Para los comentarios negativos, aplicar modelado de tópicos.
 - **Condición especial:** Si la partición (positiva o negativa) es demasiado pequeña (por debajo de un umbral definido por el estudiante), se usará un enfoque basado en **frecuencia de palabras** (nubes de palabras o listados de términos más frecuentes) en lugar de modelado de tópicos.
5. **Extracción y presentación de resultados clave:**
- Mostrar el reporte generado:
 - Las palabras clave más representativas de cada tópico identificado (positivo y negativo).
 - Al menos **un comentario representativo** por cada tópico (ej. el comentario más cercano al centroide del tópico).
6. **Visualizaciones interactivas (scatter plots):**
- Generar un **gráfico de dispersión interactivo** donde cada punto sea un comentario, coloreado (use codificación redundante) según su tópico (positivo y negativo por separado o en un mismo gráfico con distinción).
 - Los puntos deben permitir explorar el contenido del comentario al hacer hover o clic.
7. **Análisis específico del concepto "Precio, valor, costo":**

- Identificar comentarios relacionados con el concepto sintético "**precio / valor / costo**" (usando similitud semántica).
- Generar un **scatter plot interactivo** que muestre la relación entre todos los comentarios y este concepto.
- Mostrar los comentarios más relevantes a dicho concepto (ej. top 5 más cercanos por similitud).

Requerimientos Técnicos y de Entrega

- **Gestión del código:**
 - El script completo deberá alojarse en un repositorio **GitHub**.
 - El repositorio deberá contener un archivo **requirements.txt** que permita instalar todas las dependencias necesarias mediante **pip install -r requirements.txt**.
 - El repositorio deberá incluir un archivo **README.md** con instrucciones claras de instalación, configuración y uso del script (incluyendo ejemplos de ejecución con todos los parámetros).
- **Portabilidad y limitaciones:**
 - **No se permite el uso de APIs externas** (como OpenAI, Google Cloud, etc.). El script debe ejecutarse completamente offline una vez instaladas las dependencias.
- **Demostración y evaluación:**
 - Cada miembro del equipo deberá explicar su participación en el desarrollo del script (roles: preprocesamiento, análisis de sentimientos, modelado de tópicos, visualización, documentación, etc.).
 - Durante la presentación, el equipo deberá **ejecutar en vivo** el script sobre un conjunto de datos no visto previamente (o proporcionado por el profesor), demostrando que cumple con todos los puntos requeridos.

Criterios de Calificación

Criterio	Ponderación
Correcta implementación del CLI con los 5 parámetros	10%
Limpieza y preprocesamiento adecuado	10%
Análisis de outliers y n-gramas	10%
Análisis de sentimientos	15%
Modelado de tópicos / frecuencia de palabras	15%
Visualizaciones interactivas (scatter plots)	15%
Análisis específico "Precio/Valor/Costo"	10%
Calidad del repositorio GitHub (README, requirements.txt, estructura)	10%
Demostración en vivo y participación del equipo	5%

Ejemplos de repositorios/scripts de la generación pasada

1. [RodriDataScientist/pipeline_nlp](#): Este proyecto implementa un pipeline completo de análisis de texto...
2. <https://github.com/rafaelfc100/PNL-Pipeline-Generador-de-Reportes-Interactivos>
3. https://github.com/jlgc10/Pipeline_NLP_Automatico
4. <https://github.com/elica2/NLP-Analysis-Pipeline>
5. <https://github.com/Cami-au/Sript-NLP.git>
6. <https://github.com/Danie0804/An-lisis-de-Texto-y-Generaci-n-de-Reportes-HTML>